

DEVOPS 60 DAYS

ZERO TO HERO SERIES

DAY 1

PAGE 18 OF 30

TOPIC : BASIC LINUX COMMANDS - PART 12

Every command
you learn today,
saves time tomorrow!

Keep Practicing !



1. File Compression Commands

Command	Description	Example
<code>tar -cvf file.tar dir/</code>	Create tar archive	<code>tar -cvf app.tar app/</code>
<code>tar -xvf file.tar</code>	Extract tar archive	<code>tar -xvf app.tar</code>
<code>tar -czvf file.tar.gz dir/</code>	Create tar.gz archive	<code>tar -czvf app.tar.gz app/</code>
<code>tar -xzvf file.tar.gz</code>	Extract tar.gz archive	<code>tar -xzvf app.tar.gz</code>
<code>tar -cjvf file.tar.bz2 dir/</code>	Create tar.bz2 archive	<code>tar -cjvf app.tar.bz2 app/</code>
<code>tar -xjvf file.tar.bz2</code>	Extract tar.bz2 archive	<code>tar -xjvf app.tar.bz2</code>
<code>zip -r file.zip dir/</code>	Create zip archive	<code>zip -r app.zip app/</code>
<code>unzip file.zip</code>	Extract zip archive	<code>unzip app.zip</code>

Tip: Use tar.gz for backups and transfers.
Smaller size and widely compatible.

2. Disk Usage Commands

Command	Description	Example
<code>df -h</code>	Disk free (human readable)	<code>df -h</code>
<code>du -sh dir</code>	Size of directory	<code>du -sh /var/log</code>
<code>du -h --max-depth=1</code>	Size of current dir	<code>du -h --max-depth=1</code>
<code>ncdu</code>	Interactive disk usage	<code>ncdu /var</code>
<code>lsblk</code>	List block devices	<code>lsblk</code>
<code>mount</code>	Show mounted filesystems	<code>mount</code>
<code>umount /dev/sda1</code>	Unmount filesystem	<code>umount /dev/sda1</code>

Tip: ncdu is a powerful tool to find large folders easily.

3. Search & Find Commands

Command	Description	Example
<code>find / -name file.txt</code>	Find file by name	<code>find / -name file.txt</code>
<code>find / -type d -name logs</code>	Find directory by name	<code>find / -type d -name logs</code>
<code>find / -mtime N</code>	Find files modified N days ago	<code>find / -mtime 7</code>
<code>locate filename</code>	Locate file quickly	<code>locate passwd</code>
<code>updatedb</code>	Update locate database	<code>sudo updatedb</code>
<code>which python3</code>	Location of command	<code>which python3</code>
<code>whereis java</code>	Find binary, source, man	<code>whereis java</code>

Note: locate is faster than find, but database must be updated.

4. Network & Connectivity

Command	Description	Example
<code>ip a</code>	Show IP addresses	<code>ip a</code>
<code>ip r</code>	Show routing table	<code>ip r</code>
<code>ss -tln</code>	List listening ports	<code>ss -tln</code>
<code>netstat -tln</code>	List listening ports (old)	<code>netstat -tln</code>
<code>ping google.com</code>	Check connectivity	<code>ping google.com</code>
<code>traceroute google.com</code>	Trace route to host	<code>traceroute google.com</code>
<code>curl google.com</code>	Test URL/HTTP response	<code>curl google.com</code>
<code>dig google.com</code>	DNS query	<code>dig google.com</code>
<code>nslookup google.com</code>	DNS lookup	<code>nslookup google.com</code>

Tip: ss is faster and modern replacement for netstat.

5. Process & Job Management

Command	Description	Example
<code>ps aux</code>	Show all processes	<code>ps aux</code>
<code>ps -ef</code>	Full format process list	<code>ps -ef</code>
<code>top</code>	Real-time process monitor	<code>top</code>
<code>htop</code>	Interactive process viewer	<code>htop</code>
<code>kill PID</code>	Kill process by PID	<code>kill 1234</code>
<code>kill -9 PID</code>	Force kill process	<code>kill -9 1234</code>
<code>jobs</code>	List background jobs	<code>jobs</code>
<code>bg %1</code>	Resume job in background	<code>bg %1</code>
<code>fg %1</code>	Bring job to foreground	<code>fg %1</code>
<code>nice -n 10 cmd</code>	Run command with priority	<code>nice -n 10 sleep 60</code>

Tip: Use top or htop to find heavy processes.

6. Logs & Monitoring

Command	Description	Example
<code>tail -f file</code>	Follow log file in real time	<code>tail -f /var/log/syslog</code>
<code>less file</code>	View file page by page	<code>less /var/log/syslog</code>
<code>journalctl</code>	View systemd logs	<code>journalctl -xe</code>
<code>dmesg</code>	Kernel ring buffer logs	<code>dmesg less</code>
<code>free -h</code>	Memory usage	<code>free -h</code>
<code>vmstat 1 5</code>	System performance stats	<code>vmstat 1 5</code>
<code>iostat -x 1 5</code>	Disk I/O stats	<code>iostat -x 1 5</code>

Example (tail -f):

```
[devops@server ~]$ tail -f /var/log/syslog
May 24 10:30:10 server sshd[1234]: Accepted password
May 24 10:30:11 server CRON[2345]: (root) CMD (run-parts)
```

7. File Transfer Commands

Command	Description	Example
<code>scp file user@host:/path</code>	Secure copy file	<code>scp file.txt user@host:/tmp</code>
<code>scp -r dir user@host:/path</code>	Secure copy folder	<code>scp -r app/ user@host:/backup</code>
<code>rsync -avz src dest</code>	Sync files/folders	<code>rsync -avz /data/ /backup/</code>
<code>sftp user@host</code>	Secure file transfer	<code>sftp user@host</code>
<code>wget url</code>	Download file from URL	<code>wget https://site.com/file</code>
<code>curl -O url</code>	Download file	<code>curl -O https://site.com/file</code>

Example (rsync):

```
rsync -avz /var/www/ user@host:/var/www/backup/
# sync website files to remote server
```

8. Practice for Today

- ✓ Create a tar.gz archive of /etc directory.
- ✓ Find the 5 largest files in /var/log.
- ✓ Check disk usage of /home.
- ✓ Find files modified in last 7 days.
- ✓ Monitor running processes using top.
- ✓ Check open ports using ss command.
- ✓ Transfer a file to another server using scp.
- ✓ Install nginx using apt and start the service.



Challenge

Find and list the 5 largest files in /var/log and compress them into a tar.gz archive.



DEVOPS 60 DAYS

ZERO TO HERO SERIES

DAY 1

PAGE 19 OF 30

TOPIC : BASIC LINUX COMMANDS - PART 13

Small steps daily,
Big skills for life!

Keep Practicing !



1. User & Group Commands - Advanced

Command	Description	Example
useradd -m user	Create user with home dir	useradd -m devops
usermod -aG group user	Add user to supplementary group	usermod -aG sudo devops
usermod -l newuser user	Rename user	usermod -l admin devops
groupadd group	Create a group	groupadd admins
groupmod -n new-group group	Rename group	groupmod -n ops admins
gpasswd -a user group	Add user to group	gpasswd -a devops admins
gpasswd -d user group	Remove user from group	gpasswd -d devops admins
id user	Show user & group IDs	id devops

Tip: Use id to quickly see UID, GID and groups of a user.

2. File Permissions - Advanced

Command	Description	Example
chmod u+rwx file	Owner rwx	chmod u+rwx script.sh
chmod g+rx file	Group rx	chmod g+rx script.sh
chmod o-rwx file	Others remove rwx	chmod o-rwx script.sh
chmod 755 file	rwx-r-x	chmod 755 script.sh
chmod 644 file	rw-r--	chmod 644 file.txt
chown user file	Change owner	chown devops file.txt
chown user:group file	Change owner & group	chown devops:admins file.txt
chgrp group file	Change group	chgrp admins file.txt

Tip: 7 = rwx, 5 = r-x, 4 = r-- ; Use numbers for quick changes.

3. File Searching - Advanced

Command	Description	Example
find /path -name "*.log"	Find files by name	find /var/log -name "*.log"
find /path -type d	Find directories	find /etc -type d
find /path -size +100M	Find files larger than size	find / -size +100M
find /path -mtime -7	Modified in last 7 days	find /home -mtime -7
locate filename	Locate file quickly	locate passwd
updatedb	Update locate database	sudo updatedb

Note: updatedb must be run before using locate.

4. Text Processing - Advanced

Command	Description	Example
grep -i "error" file	Case-insensitive search	grep -i "error" app.log
grep -r "error" /var/log	Recursive search	grep -r "error" /var/log
awk '{print \$1}' file	Print first column	awk '{print \$1}' data.txt
awk -F: '{print \$NF}' file	Print last field	awk -F: '{print \$NF}' /etc/passwd
cut -d',' -f1 file	Extract first column	cut -d',' -f1 data.csv
sort file.txt	Sort lines	sort names.txt
sort -u file.txt	Sort unique lines	sort -u names.txt
uniq file.txt	Remove duplicate lines	uniq names.txt
wc -l file.txt	Count lines	wc -l file.txt
wc -w file.txt	Count words	wc -w file.txt
wc -c file.txt	Count characters	wc -c file.txt

Tip: grep, awk, sed are very powerful text tools.

5. Process Monitoring - Advanced

Command	Description	Example
ps -eo pid,ppid,cmd,%mem,%cpu	Custom process format	ps -eo pid,ppid,cmd,%mem,%cpu
pstree -p	Process tree with PID	pstree -p
top -b -n 1	Batch mode top output	top -b -n 1 > out.txt
htop	Interactive process viewer	htop
watch -n 2 ps aux	Repeat command	watch -n 2 ps aux
kill -STOP PID	Pause process	kill -STOP 1234
kill -CONT PID	Resume process	kill -CONT 1234
nice -n 10 cmd	Run with lower priority	nice -n 10 command
renice 5 -p PID	Change priority	renice 5 -p 1234

Tip: Use htop for better and colored process view.

6. Disk & Filesystem - Advanced

Command	Description	Example
df -Th	Disk usage with filesystem type	df -Th
du -ah /path sort -h	Disk usage of files in human readable	du -ah /home sort -h
ncdu /path	Interactive disk usage	ncdu /var
lsblk -f	Block devices with filesystem	lsblk -f
fdisk -l	List partition table	fdisk -l
mount /dev/sdb1 /mnt	Mount device	mount /dev/sdb1 /mnt
umount /mnt	Unmount device	umount /mnt
fsck /dev/sdb1	Check filesystem	fsck /dev/sdb1

Tip: Always unmount before running fsck.

7. Network Commands - Advanced

Command	Description	Example
ip -br addr	Short IP address	ip -br addr
ip route show	Show routing table	ip route show
ss -tuln	List listening ports	ss -tuln
netstat -tulnp	List listening ports with PID	netstat -tulnp
ping -c 4 host	Ping with count	ping -c 4 google.com
traceroute host	Trace route	traceroute google.com
dig domain	DNS query	dig google.com
nslookup domain	DNS lookup	nslookup google.com
curl -I url	HTTP headers only	curl -I https://site.com

Tip: ss is faster and more powerful than netstat.

8. Practice for Today

- ✓ Create a user and add to a group.
- ✓ Change file permissions using numbers.
- ✓ Find all .log files modified in last 7 days.
- ✓ Find top 5 large files in /var.
- ✓ Check running processes using htop.
- ✓ Check listening ports using ss command.
- ✓ Transfer a file to another server using scp.
- ✓ Monitor disk usage using df -Th.
- ✓ Compress /etc directory into backup tgz.
- ✓ Install a package using apt or yum.

Challenge

Find the top 5 processes using CPU and memory and list them.



DEVOPS 60 DAYS

ZERO TO HERO SERIES

DAY 1

PAGE 20 OF 30

TOPIC : ADVANCED LINUX SHELL SCRIPTING BASICS

Learn a little today,
Automate tomorrow,
Master forever!

Keep Practicing !



1. Variables

Type	Syntax	Example	Output
Assign	var=value	name=DevOps	-
Use	\$var	echo \$name	DevOps
Read Only	readonly var	readonly PI=3.14	-
Environment	export var=value	export PATH=\$PATH:/opt/bin	-
Unset	unset var	unset name	-

💡 Tip: No spaces around = while assigning value.

2. User Input (Read)

Syntax	Example
read var	read name
read -p "Msg: " var	read -p "Enter Age: " age
read -s var (silent)	read -s password

💡 Tip: Use -s for sensitive input like passwords.

3. Output / Display

Command	Use	Examples
echo	Print text to screen	echo "Hello DevOps"
echo -n	Print without newline	echo -n "Name: "
printf	Formatted output	printf "Age: %d\n" 25
cat file	Display file content	cat /etc/os-release
tput / clear	Terminal control	tput setaf 2; echo Green

💡 Tip: printf is better for formatting than echo.

4. Conditions (Decision Making)

Type	Syntax	Example
If	if [condition]; then commands fi	if [\$age -ge 18]; then echo "Adult" fi
If-Else	if [condition]; then commands else commands fi	if [-f file.txt]; then echo "Found" else echo "Not Found" fi
Else If	if [cond1]; then commands elif [cond2]; then commands else commands fi	if [\$n -gt 10]; then echo "Big" elif [\$n -eq 10]; then echo "Ten" else echo "Small" fi

Common Operators

-eq	Equal	-ge	Greater or equal
-ne	Not Equal	-le	Less or equal
-gt	Greater than	=	String equal
-lt	Less than	!=	String not equal

💡 Tip: Use spaces inside [and around operators.

5. Loops

Type	Syntax	Example
For Loop	for i in item1 item2 ...; do commands done	for i in 1 2 3; do echo \$i done
While Loop	while [condition]; do commands done	count=1 while [\$count -le 5]; do echo \$count count=\$((count+1)) done
Until Loop	until [condition]; do commands done	n=1 until [\$n -gt 5]; do echo \$n n=\$((n+1)) done
C-Style For	for ((init; condition; update)); do commands done	for ((i=1; i<=5; i++)); do echo \$i done

💡 Tip: Use loops to automate repetitive tasks.

6. Functions

Syntax	Example	Notes
function_name() { commands return status } function_name # call function	greet() { echo "Hello \$1" return 0 } greet "DevOps"	• \$1, \$2 ... are arguments. • return 0 = Success return non-zero = Failure

💡 Tip: Functions help reuse code and keep scripts clean.

7. Exit Codes

Code	Meaning	Example
0	Success	ls /no/such/file
1	General Error	echo \$? # Output: 2
2	Misuse of builtins	false
126	Command invoked cannot execute	echo \$? # Output: 1
127	Command not found	
130	Script terminated by Ctrl+C	

💡 Tip: Always check exit status using echo \$?

8. Shebang

Syntax	Common Shebangs
#!/bin/bash # or #!/bin/sh # First line of script	#!/bin/bash -> Bash shell #!/bin/sh -> POSIX shell #!/usr/bin/env bash -> Portable bash

💡 Tip: Shebang tells the system which interpreter to use.

9. Best Practices

- ✓ Use meaningful variable and function names.
- ✓ Always quote variables: "\$var"
- ✓ Add comments for explanation.
- ✓ Use set -e to exit on error.
- ✓ Use shellcheck tool to find issues in script.
- ✓ Handle errors and validate inputs.
- ✓ Keep scripts modular and reusable.

Challenge

Write a script that takes a directory path and lists number of files, directories and size.



DEVOPS 60 DAYS

ZERO TO HERO SERIES

DAY 1

PAGE 21 OF 30

TOPIC : ADVANCED LINUX SHELL SCRIPTING BASICS

Small steps daily,
Big skills for life!

Keep Practicing !



1. Variables - More Examples

Type	Syntax	Example	Output / Notes
Integer	a=10	a=10	Number
Float	pi=3.14	pi=3.14	Floating point
String	name="DevOps"	name="DevOps"	Text value
Readonly	readonly x=100	x=100	Cannot change
Multiple	a=1; b=2	a=1, b=2	Multiple vars
Default	\${var:-default}	\${name:-guest}	If name not set
Length	\${#var}	len=\${#name}	Length of value

Tip: Always quote variables: "\$var"
e.g. echo "\$name" (safer for spaces)

2. User Input - More Examples

Syntax	Example
read n	read n; echo "Name: \$n"
read -p "Enter Age: " age	read -p "Enter Age: " age
read -s pass	read -s pass; echo
read -p "Confirm (y/n): " ans	read -p "Confirm (y/n): " ans
IFS=' ' read -r a b c	IFS=' ' read -r a b c

Tip: Use read -r to prevent backslash interpretation.

3. Output / Redirection

Type	Syntax	Example	Description
Standard Output	command > file	ls > out.txt	Overwrite output
Append Output	command >> file	echo "Hi" >> out.txt	Append output
Error Output	command 2> err.log	cat nofile 2> err.log	Redirect errors
Both Output	command &> file	ls /no &> all.log	Redirect all
Pipe	cmd1 cmd2	ps aux grep nginx	Pass output
Pipe to File	cmd tee file	du -sh / tee log.txt	Output to file & screen

Tip: Use > file (overwrite) and >> file (append) carefully.

4. Conditions - Advanced

Type	Syntax	Example	Meaning
File exists	[-e file]	[-e /etc/passwd]	Check existence
File not exists	[! -e file]	[! -e /tmp/a]	Not exists
Directory	[-d dir]	[-d /var/log]	Is a directory
File	[-f file]	[-f my.sh]	Is a regular file
Readable	[-r file]	[-r my.sh]	Readable
Writable	[-w file]	[-w my.sh]	Writable
Executable	[-x file]	[-x my.sh]	Executable
String empty	[-z "\$var"]	[-z "\$name"]	String is empty
String not empty	[-n "\$var"]	[-n "\$name"]	String not empty
String equal	["\$a" = "\$b"]	["\$a" = "\$b"]	Equal
String not equal	["\$a" != "\$b"]	["\$a" != "\$b"]	Not equal
Number equal	[\$a -eq \$b]	[5 -eq 5]	Numbers equal
Number greater	[\$a -gt \$b]	[10 -gt 5]	Greater than
Number less	[\$a -lt \$b]	[3 -lt 7]	Less than

Tip: Use [[] instead of [] for complex expressions.
e.g. if [[\$a -gt 5 && -f file]]; then ... fi

5. Loops - More Examples

Type	Syntax	Example	Output
For Loop	for i in 1 2 3; do echo \$i; done	for i in a b c; do echo "\$i"; done	1 2 3 or a b c
For File Loop	for f in *; do echo \$f; done	for f in /etc/*.conf; do echo \$f; done	List all .conf files in /etc
C-Style For	for ((i=1; i<=3; i++)); do echo \$i; done	for ((i=0; i<=5; i++)); do echo \$i; done	0 1 2 3 4
While Loop	i=1 while [\$i -le 3]; do echo \$i; i=\$((i+1)); done	count=1 while [\$count -le 3]; do echo \$count; count=\$((count+1)); done	1 2 3
Until Loop	i=1 until [\$i -gt 3]; do echo \$i; i=\$((i+1)); done	n=1 until [\$n -gt 3]; do echo \$n; n=\$((n+1)); done	1 2 3

Tip: Use loops to avoid repeating the same commands.

6. Functions - Advanced

Syntax	Example	Output
function-name() { # commands return status }	sum() { a=\$1; b=\$2 c=\$((a+b)) echo \$c return 0 } sum 10 20 echo "Sum: \$?"	30 Sum: 0

Tip: Functions improve reusability and readability.

7. Exit Codes - Details

Code	Meaning	Use Case
0	Success	Command executed successfully
1	General Error	Generic error
2	Misuse of shell builtins	Wrong usage of command
126	Permission denied	File found but not executable
127	Command not found	Command not available in PATH
128	Invalid argument to exit	Invalid exit argument
130	Script terminated by Ctrl+C	User interrupted
255	Exit status out of range	Invalid exit code

Tip: Always check with echo \$? after commands.

8. Shebang - Details

Common Shebangs	Use Case
#!/bin/bash	-> Bash shell
#!/bin/sh	-> POSIX shell
#!/usr/bin/env bash	-> Portable bash
#!/bin/ksh	-> Korn shell
#!/bin/zsh	-> Z shell

Use #!/usr/bin/env bash for better portability across systems.

Tip: Shebang tells the system which interpreter to use.

9. Best Practices

- ✓ Use meaningful variable and function names.
- ✓ Always quote variables: "\$var".
- ✓ Add comments for explanation.
- ✓ Use set -e to exit on error.
- ✓ Use shellcheck to find issues in scripts.
- ✓ Validate inputs and handle errors.
- ✓ Keep scripts modular and reusable.
- ✓ Test your scripts in non-production first.



Challenge

Write a script that asks a number N and prints table from 1 to 10 of N using a loop.



DEVOPS 60 DAYS

ZERO TO HERO SERIES

DAY 1

PAGE 22 OF 30

TOPIC : ADVANCED LINUX SHELL SCRIPTING BASICS

Small steps daily,
Big skills for life!

Keep Practicing !



1. Variables - More Details

Type	Syntax	Example	Output / Notes
Local Variable	name=value	count=10	Available in current shell
Environment	export name=value	export PATH=/usr/bin	Available in child processes
Read-Only	readonly name	readonly PI=3.14	Cannot be changed
Unset Variable	unset name	unset count	Removes variable

💡 Tip: Use \${var} to access variables.
Example: echo \${name}

2. Command Line Arguments (\$0, \$1, ..., \$@, \$#)

Parameter	meaning	Example
\$0	Script name	\$0 → script.sh
\$1, \$2 ...	First, second argument	\$1 → apple, \$2 → mango
\$#	Number of arguments	\$# → 3
\$@	All arguments	\$@ → apple mango orange
\$*	All arguments as single string	* → apple mango orange

💡 Tip: Always quote \$@ and \$* like "\$@" to handle spaces.

3. Input from User (Interactive)

Syntax	Example
read var	read name
read -p "Enter age: " age	read -p "Enter age: " age
read -s pass	read -s pass (hidden input)
read -p "Continue? (y/n): " ans	read -p "Continue? (y/n): " ans

💡 Tip: Use -s for passwords, -p for prompts.

4. Conditional Statements - Advanced

Type	Syntax	Example
if	if [condition]; then commands fi	if [\$age -ge 18]; then echo "Adult" fi
if-elif-else	if [cond1]; then commands elif [cond2]; then commands else commands fi	if [\$n -gt 10]; then echo "Big" elif [\$n -eq 10]; then echo "Ten" else echo "Small" fi
case	case \$var in pattern1) commands ;; pattern2) commands ;; *) commands ;; esac	case \$ch in [yY]) echo "Yes" ;; [nN]) echo "No" ;; *) echo "Invalid" ;; esac

Common Test Operators	String / Numeric Operators
File Operators	-z String length is zero
-e Exists	-n String length is non-zero
-f Regular File	= Equal
-d Directory	!= Not equal
-r Readable	-eq Equal (numbers)
-w Writable	-ne Not equal
-x Executable	-gt Greater than
-s Size > 0	-lt Less than
	-ge Greater or equal
	-le Less or equal

💡 Tip: Add spaces around [and operators.
Example: ["\$a" -eq 5] (correct)

5. Loops - More Details

Loop Type	Syntax	Example
For Loop	for i in item1 item2 ... ; do commands done	for i in 1 2 3; do echo "i = \$i"; done
While Loop	while [condition]; do commands done	count=1 while [\$count -le 5]; do echo \$count count=\$((count+1)) done
Until Loop	until [condition]; do commands done	n=1 until [\$n -gt 5]; do echo \$n n=\$((n+1)) done
C-Style For	for ((init; condition; update)); do commands done	for ((i=1; i<=5; i++)); do echo \$i done

💡 Tip: Prefer for loop for lists and while for conditions.

6. Functions - More Details

```
function_name() {  
    # local variables  
    local a=$1  
    local b=$2  
    sum=$((a+b))  
    echo $sum  
    return 0  
}  
# Call  
result=$(function_name 5 10)  
echo "Sum: $result"
```

Notes

- Use `local` to keep variables local to function.
- `return 0` → success
 `return non-zero` → failure
- Output can be captured using command substitution.

💡 Tip: Functions make scripts modular and reusable.

7. Exit Codes - Complete List

Code	Meaning	Use Case / Notes
0	Success	Command executed successfully
1	General Error	Generic error
2	Misuse of shell builtins	Wrong usage of command
126	Permission denied	File found but not executable
127	Command not found	Command not available in PATH
128	Invalid argument to exit	Invalid exit argument
138	Script terminated by Ctrl+C	User interrupted
255	Exit status out of range	Invalid exit code

💡 Tip: Check exit status with `echo $?` after every command.

8. Shebang & Portability

Shebang Examples

```
#!/bin/bash → Bash shell  
#!/bin/sh → POSIX shell  
#!/usr/bin/env bash → Portable bash  
#!/usr/bin/env sh → Portable sh
```

Portability Tips

- Use `/usr/bin/env` for portability.
- Avoid bash-specific features in sh.
- Stick to POSIX compatible syntax.

💡 Tip: Shebang tells the system which interpreter to use.

9. Best Practices for Shell Scripts

- ☑ Use meaningful variable and function names.
- ☑ Always quote variables: "\$var".
- ☑ Add comments for explanation.
- ☑ Use `set -e` to exit on error.
- ☑ Use shellcheck to find issues in scripts.
- ☑ Handle errors and validate inputs.
- ☑ Keep scripts modular and reusable.
- ☑ Test your scripts in non-production first.
- ☑ Follow consistent indentation and spacing.

Challenge

Write a script that accepts N numbers and prints sum, average, max and min using loops and functions.



DEVOPS 60 DAYS

ZERO TO HERO SERIES

DAY 1

PAGE 23 OF 30

TOPIC : ADVANCED LINUX SHELL SCRIPTING BASICS

Small steps daily,
Big skills for life!

Keep Practicing !



1. Special Variables

Variable	Meaning	Example
\$0	Script name	\$0 → backup.sh
\$1, \$2 ...	Positional parameters	\$1 → first arg
\$#	Number of arguments	\$# → 3
\$@	All arguments	\$@ → "a b c"
\$*	All arguments as single string	\$* → "a b c"
\$\$	PID of current shell	\$\$ → 12345
\$?	Exit status of last command	\$? → 0
\$_	PID of last background job	\$_ → 23456
\$_	Last argument of previous cmd	\$_ → file.txt

Tip: \$@ is preferred in loops to handle arguments safely.

2. File Test Operators

Operator	Meaning	Example
-e file	File exists	[-e file.txt]
-f file	Regular file	[-f file.txt]
-d dir	Directory exists	[-d /home/user]
-s file	File is not empty	[-s file.txt]
-r file	Readable	[-r file.txt]
-w file	Writable	[-w file.txt]
-x file	Executable	[-x script.sh]

Tip: Always quote variables: [-f "\$file"]

3. String Operations

Operation	Syntax	Example	Output
Length	\${#var}	\${#name}	5
Substring	\${var:pos}	\${name:1}	evelops
Substring	\${var:pos:len}	\${name:1:4}	evel
Replace	\${var/old/new}	\${name/Dev/Ops}	Opsvelops
Replace All	\${var/old/new}	\${path//./-}	usr-bin

Tip: Index starts from 0.

4. Arithmetic Operations

Operation	Syntax	Example	Output
Add	=\$((a + b))	=\$((5 + 3))	8
Sub	=\$((a - b))	=\$((5 - 3))	2
Mul	=\$((a * b))	=\$((5 * 3))	15
Div	=\$((a / b))	=\$((10 / 2))	5
Mod	=\$((a % b))	=\$((10 % 3))	1
Inc	=\$((a++))	a=5;=\$((a++))	5 then 6
Dec	=\$((a--))	a=5;=\$((a--))	5 then 4

Tip: Use \$(()) for arithmetic, not expr or bc for simple math.

5. Case Statement (Switch)

```
case $var in
  start)
    echo "Starting service" ;;
  stop)
    echo "Stopping service" ;;
  restart)
    echo "Restarting service" ;;
  *)
    echo "Usage: start|stop|restart";;
esac
```

Example:

```
./script.sh start
```

Output:

Starting service

Tip: case is cleaner than multiple if-elif blocks.

6. Advanced Loop Examples

Loop Type	Syntax	Example	Output
For Loop (Sequence)	for i in {1...5}; do echo \$i; done	for i in {1...5}; do echo \$i; done	1 2 3 4 5
For Loop (Word List)	for f in *.sh; do echo \$f; done	for f in a.sh b.sh; do echo \$f; done	a.sh b.sh
Nested Loop	for i in 1 2; do for j in a b; do echo \$i\$j; done done	for i in 1 2; do for j in a b; do echo \$i\$j; done done	1a 1b 2a 2b
Break & Continue	for i in {1...5}; do [\$i -eq 3] && continue echo \$i [\$i -eq 4] && break done	for i in {1...5}; do [\$i -eq 3] && continue echo \$i [\$i -eq 4] && break done	1 2 5

Tip: Use break to exit loop & continue to skip iteration.

7. Command Substitution

Syntax:

```
result=$(command)
```

```
echo "Date is $(date +%F)"
```

Example:

```
today=$(date +%F)
```

```
echo "Today: $today"
```

Tip: Backticks `command` also work but \$() is recommended.

8. Here Documents

```
cat <<EOF > file.txt
Hello DevOps
Today is $(date)
This is a here document.
EOF
```

Example: Creates file.txt with multiple lines.

Tip: Use quoted EOF ('EOF') to prevent variable expansion.

9. Better Script Structure

```
#!/bin/bash
```

```
# Description: Backup script
# Author: DevOps Engineer
# Date: $(date +%F)
```

```
set -euo pipefail # safer scripts
```

```
main() {
  echo "Running backup..."
  # your logic here
}
```

```
main "$@"
```

Notes:

- set -e : Exit on error
- set -u : Error on undefined variables
- set -o pipefail : Catch errors in pipelines
- Functions keep code modular.

Tip: Always use set -euo pipefail in production scripts.

10. Common Useful Commands in Scripts

Command	Use	Example
dirname	Get directory name	dirname /a/b/file.txt → /a/b
basename	Get file name	basename /a/b/file.txt → file.txt
date	Date & time	date +%F_%T → 2025-05-24_15:30:00
wc	Count lines, words, chars	wc -l file.txt
sort	Sort lines	sort file.txt
uniq	Remove duplicates	sort file.txt uniq
grep	Search pattern	grep "error" log.txt

Tip: Combine commands with pipes (|) to build powerful tools.

DEVOPS 60 DAYS

ZERO TO HERO SERIES

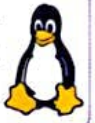
DAY 1

PAGE 24 OF 30

TOPIC : ADVANCED LINUX SHELL SCRIPTING BASICS

Small steps daily.
Big skills for life!

Keep Practicing !



1. Read File Line by Line

```
while IFS= read -r line; do
    echo "Line => $line"
done < file.txt
```

Example: file.txt
DevOps
Linux
Shell
Scripting

💡 Tip: IFS= and -r options prevent leading/trailing spaces and backslash escapes.

2. Check if File Exists

Condition	Description	Example
-e file	File exists	[-e myfile.txt]
-f file	Regular file exists	[-f myfile.txt]
-d dir	Directory exists	[-d mydir]
-r file	Readable	[-r myfile.txt]
-w file	Writable	[-w myfile.txt]
-x file	Executable	[-x script.sh]
-s file	File size > 0	[-s myfile.txt]

💡 Tip: Always quote variables: [-f "\$file"]

3. Array Basics

```
# Declare array
names=(DevOps Linux Shell Bash)
echo "First: ${names[0]}"
echo "All: ${names[@]}"
echo "Count: ${#names[@]}"
```

Output:
First: DevOps
All: DevOps Linux Shell Bash
Count: 4

💡 Tip: Arrays are zero-indexed.

4. Using Select (Menu)

```
select opt in Start Stop Status Exit; do
    case $opt in
        Start) echo "Starting..." ;;
        Stop)  echo "Stopping..." ;;
        Status) echo "Running" ;;
        Exit) break ;;
        *) echo "Invalid option" ;;
    esac
done
```

Sample Run:

```
1) Start
2) Stop
3) Status
4) Exit
#? 3
Running
```

💡 Tip: select provides a simple numbered menu.

5. Case Statement - More Example

```
read -p "Enter your choice (start/stop/restart):" ch
case $ch in
    start) echo "Service started" ;;
    stop)  echo "Service stopped" ;;
    restart) echo "Service restarted" ;;
    *) echo "Invalid choice" ;;
esac
```

Test:
start -> Service started
stop -> Service stopped
restart -> Service restarted
other -> Invalid choice

💡 Tip: Case is cleaner and faster than multiple if-else.

6. Nested Loops Example

```
for i in 1 2; do
    for j in a b; do
        echo "i=$i j=$j"
    done
done
```

Output:

```
i=1 j=a
i=1 j=b
i=2 j=a
i=2 j=b
```

💡 Tip: Indentation improves readability of nested loops.

7. Functions with Arguments

```
greet() {
    echo "Hello $1, Welcome to $2!"
}

# Call
greet "Alice" "DevOps"
```

Output:

```
Hello Alice,
Welcome to DevOps!
```

💡 Tip: Inside function use \$1, \$2 ... for arguments.

8. Exit Codes in Scripts

Code	Meaning	When to Use
0	Success	Script completed successfully
1	General Error	Something went wrong
2	Misuse of shell builtins	Wrong usage of command
126	Permission denied	File exists but not executable
127	Command not found	Command not available in PATH
130	Script terminated by Ctrl+C	User interrupted
255	Exit status out of range	Invalid exit code

💡 Tip: Use meaningful exit codes for troubleshooting.

9. Here Document Examples

```
cat <<EOF
Hello DevOps Engineer
Today is $(date)
We love automation!
EOF
```

Output:

```
Hello DevOps Engineer
Today is Tue May 14 15:30:00 IST 2025
We love automation!
```

💡 Tip: Use quoted EOF ('EOF') to prevent variable expansion.

10. Best Practices

- ✓ Always use #!/bin/bash as first line.
- ✓ Use set -e to exit on error.
- ✓ Use set -u to treat unset variables as error.
- ✓ Use set -o pipefail for safer pipelines.
- ✓ Always quote variables: "\$var"
- ✓ Use meaningful variable and function names.
- ✓ Add comments for explanation.
- ✓ Validate inputs and handle errors.
- ✓ Keep scripts modular and reusable.
- ✓ Test your scripts in non-production first.

★ Challenge

Write a script that accepts a directory path and lists all .log files, counts lines in each and shows total size.



DEVOPS 60 DAYS

ZERO TO HERO SERIES

DAY 1

PAGE 25 OF 30

TOPIC : ADVANCED LINUX SHELL SCRIPTING BASICS

Small steps daily.
Big skills for life!

Keep Practicing !



1. Trap with Signals - Advanced

Signal	Number	Meaning	Example Use
SIGINT	2	Interrupt (Ctrl+C)	Graceful exit
SIGTERM	15	Terminate	Cleanup before exit
EXIT	-	Script exiting	Final cleanup

Tip: Use 'trap' to catch signals and perform cleanup.

2. Debugging with set -x and set -v

```
#!/bin/bash
set -x # Print commands
set -v # Print input lines
echo "Debugging started..."
VAR=10
echo "Value is $VAR"
set +x # Disable xtrace
set +v # Disable verbose
```

Output (Example)

```
+ VAR=10
+ echo "Value is 10"
Value is 10
```

Tip: Remove debugging options in production scripts.

3. Command Line Arguments - Advanced

Parameter	Meaning	Example
\$0	Script name	\$0 → script.sh
\$1 ... \$9	First 9 arguments	\$1 → apple
\$#	Total number of args	\$# → 3
\$@	All arguments as list	\$@ → a b c
\$*	All arguments as string	* → "a b c"
\$\$	Current PID	\$\$ → 12345
\$?	Last exit code	\$? → 0

Tip: Use "\$@" to handle arguments with spaces.

4. Here Document (Expansion Control)

```
cat <<'EOF' # No variable expansion
Hello $USER
Today is $(date)
EOF
```

Output

```
Hello $USER
Today is $(date)
```

Tip: Use quotes around EOF to prevent expansion.

5. Arithmetic Expansion

```
a=10; b=3
sum=$((a + b))
mul=$((a * b))
div=$((a / b))
mod=$((a % b))
echo "Sum: $sum, Mul: $mul,
      Div: $div, Mod: $mod"
```

Output

```
Sum: 13, Mul: 30,
Div: 3, Mod: 1
```

Tip: Use \$(()) for arithmetic, not expr.

6. Select Loop (Menu Driven)

```
#!/bin/bash
select opt in Start Stop Restart Exit
do
    case $opt in
        Start) echo "Starting..." ;;
        Stop)  echo "Stopping..." ;;
        Restart) echo "Restarting..." ;;
        Exit)  break ;;
        *)     echo "Invalid option" ;;
    esac
done
```

Sample Run

```
1) Start
2) Stop
3) Restart
4) Exit
#? 2
Stopping...
```

Tip: Great for CLI menus and admin tools.

7. Arrays - Advanced Operations

```
fruits=(apple mango banana)
fruits+=(grapes orange)
echo "All: ${fruits[@]}"
echo "Count: ${#fruits[@]}"
unset fruits[2] # remove banana
echo "After unset: ${fruits[@]}"
```

Output

```
All: apple mango banana
      grapes orange
Count: 5
After unset:
apple mango grapes orange
```

Tip: Arrays are powerful for lists and bulk operations.

8. Read with Options

```
read -r -p "Enter name: " name
read -s -p "Enter password: " pass
echo
echo "Name: $name"
echo "Password length: ${#pass}"
```

Note

```
-r → raw input
-s → silent input
    (no echo)
```

Tip: Use -r and -s for safer input handling.

9. Case Statement - Advanced Patterns

```
read -p "Enter OS (linux/windows/mac): " os
case ${os,,} in # convert to lowercase
    linux) echo "Linux selected" ;;
    windows) echo "Windows selected" ;;
    mac|darwin) echo "Mac selected" ;;
    *) echo "Unknown OS" ;;
esac
```

Output (Example)

```
Enter OS
(linux/windows/mac):
Linux selected
```

Tip: Use \${var,,} to convert to lowercase.

10. Script Best Practices (Advanced)

- Use 'set -e' and 'set -o pipefail' in production.
- Use functions and modular code.
- Validate inputs and handle errors.
- Log important actions and errors.
- Keep secrets/configs outside the script.
- Write clean, readable and maintainable code.
- Test scripts in non-production first.

Challenge

Write a script that takes username list, checks if user exists, logs to file and handles errors gracefully.



DEVOPS 60 DAYS

ZERO TO HERO SERIES

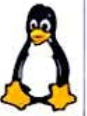
DAY 1

PAGE 26 OF 30

TOPIC : ADVANCED LINUX SHELL SCRIPTING BASICS

Small steps daily.
Big skills for life!

Keep Practicing !



1. Test - File and Directory

Option	Meaning	Example
-e file	File exists	[-e file.txt]
-f file	Regular file exists	[-f file.txt]
-d dir	Directory exists	[-d /home/user]
-r file	Readable	[-r file.txt]
-w file	Writable	[-w file.txt]
-x file	Executable	[-x script.sh]
-s file	File is not empty	[-s file.txt]
-z str	String length is zero	[-z "\$str"]
-n str	String length is non-zero	[-n "\$str"]
file1 -nt file2	file1 is newer than file2	[file1 -nt file2]
file1 -ot file2	file1 is older than file2	[file1 -ot file2]

💡 Tip: Use test or [condition] in scripts.

2. String Manipulation

Operation	Syntax	Example	Output
Length	\${#var}	str="hello" echo \${#str}	5
Substring	\${var:pos}	str="hello" echo \${str:1}	ello
Substring	\${var:pos:len}	str="hello" echo \${str:1:3}	ell
Replace	\${var/old/new}	str="hello" echo \${str/ll/pp}	heppo
Replace All	\${var//old/new}	str="banana" echo \${str//a/o}	bonono
Uppercase	\${var^^}	str="hello" echo \${str^^}	HELLO
Lowercase	\${var,,}	str="HELLO" echo \${str,,}	hello

💡 Tip: Parameter expansion makes scripts powerful!

3. Functions - Advanced

```
# Function with arguments and return value
add() {
    local a=$1
    local b=$2
    echo $((a+b)) # return via echo
}
result=$((add 10 20))
echo "Sum is $result"
```

Output:
Sum is 30

💡 Tip: Functions help break scripts into reusable blocks.

4. While and Until Loops

While Loop	Until Loop
<pre>i=1 while [\$i -le 5]; do echo "Number: \$i" i=\$((i+1)) done</pre>	<pre>i=1 until [\$i -gt 5]; do echo "Count: \$i" i=\$((i+1)) done</pre>

💡 Tip: while runs while condition is true, until runs until condition becomes true.

5. Case Statement - Advanced

```
read -p "Enter your choice (a/b/c): " choice
case $choice in
    a) echo "You chose A" ;;
    b) echo "You chose B" ;;
    c) echo "You chose C" ;;
    *) echo "Invalid choice" ;;
esac
```

Sample Run:

Enter your choice (a/b/c): b
You chose B

💡 Tip: case is cleaner than multiple if-elif-else.

6. Reading From Files

```
# Read file line by line
while IFS= read -r line; do
    echo "Line: $line"
done < sample.txt
```

sample.txt Content:

One
Two
Three

💡 Tip: IFS= and -r keep spaces and backslashes intact.

7. Command Substitution - Advanced

```
current_date=$(date +%Y-%m-%d)
uptime_info=$(uptime -p)
echo "Date: $current_date"
echo "Uptime: $uptime_info"
```

Output:

Date: 2025-05-24
Uptime: up 1 hour,
20 minutes

💡 Tip: Use \$() to capture command output.

8. Handling Arguments (Advanced)

```
if [ $# -eq 0 ]; then
    echo "Usage: $0 <file> <mode>"
    exit 1
fi
file=$1
mode=$2
echo "File: $file, Mode: $mode"
```

Example Run:

\$./script.sh data.txt read
File: data.txt, Mode: read

💡 Tip: Always validate arguments in scripts.

9. Special Variables

Variable	Meaning	Example
\$0	Script name	./script.sh
\$1...\$9	Arguments	\$1, \$2...
#\$	Number of args	3
#@	All args as list	"\$@"
#\$	All args as string	"\$*"
\$\$	PID of script	12345
\$?	Exit status of last cmd	0
\$!	PID of last background cmd	6789

💡 Tip: These variables are very useful in real scripts.

DEVOPS 60 DAYS

ZERO TO HERO SERIES

DAY 1

PAGE 27 OF 30

TOPIC : ADVANCED LINUX SHELL SCRIPTING BASICS

Small steps daily.
Big skills for life!

Keep Practicing !



1. Here Document (Multi-line Input)

```
cat <<EOF
Hello DevOps Engineer,
Welcome to Shell Scripting!
Keep learning and keep growing.
EOF
```

Output:

```
Hello DevOps Engineer,
Welcome to Shell
Scripting!
Keep learning and
keep growing.
```

💡 Tip: Here Document is great for multi-line input.

6. Function with Parameters

```
greet() {
    echo "Hello $1, Welcome to DevOps!"
}

greet "Alice"
greet "Bob"
```

Output:

```
Hello Alice,
Welcome to DevOps!

Hello Bob,
Welcome to DevOps!
```

💡 Tip: \$1, \$2, ... are positional parameters.

2. Command Substitution

```
DATE=$(date +%Y-%m-%d)
UPTIME=$(uptime -p)
echo "Date: $DATE"
echo "Uptime: $UPTIME"
```

Output:

```
Date: 2025-05-24
Uptime: up 1 hour,
20 minutes
```

💡 Tip: Use \$() to store command output in a variable.

7. Nested If - Advanced

```
read -p "Enter marks: " MARKS
if [ $MARKS -ge 90 ]; then
    echo "Grade: A"
elif [ $MARKS -ge 75 ]; then
    echo "Grade: B"
elif [ $MARKS -ge 50 ]; then
    echo "Grade: C"
else
    echo "Grade: Fail"
fi
```

Sample Run:

```
Enter marks: 82
Grade: B
```

💡 Tip: Check conditions from high to low for efficiency.

3. Read Input from User

```
echo -n "Enter your name: "
read NAME
echo "Hello, $NAME!"
```

Sample Run:

```
Enter your name: Shalom
Hello, Shalom!
```

💡 Tip: Use -n to avoid newline in prompt.

8. Case Statement - Menu Driven

```
echo "1) Start"
echo "2) Stop"
echo "3) Restart"
read -p "Enter choice: " CHOICE
case $CHOICE in
    1) echo "Starting service..." ;;
    2) echo "Stopping service..." ;;
    3) echo "Restarting service..." ;;
    *) echo "Invalid choice" ;;
esac
```

Sample Run:

```
Enter choice: 2
Stopping service...
```

💡 Tip: Case is cleaner than multiple if-elif-else.

4. Arrays - More Examples

```
NAMES=(Alice Bob Charlie David)
echo "First: ${NAMES[0]}"
echo "All: ${NAMES[@]}"
echo "Count: ${#NAMES[@]}"
echo "Last: ${NAMES[-1]}"
```

Output:

```
First: Alice
All: Alice Bob Charlie David
Count: 4
Last: David
```

💡 Tip: Negative index gives value from the end.

9. Practice Challenge

Write a script that:

- # Takes username as input
- # Checks if user exists
- # If exists, shows last login
- # Else, shows "User not found"



Challenge

Practice daily,
Become better
everyday!

5. Arithmetic Operations

```
a=10; b=3
echo "a + b = $((a + b))"
echo "a - b = $((a - b))"
echo "a * b = $((a * b))"
echo "a / b = $((a / b))"
echo "a % b = $((a % b))"
```

Output:

```
a + b = 13
a - b = 7
a * b = 30
a / b = 3
a % b = 1
```

💡 Tip: Use \$(()) for arithmetic evaluation.

DEVOPS 60 DAYS

ZERO TO HERO SERIES

DAY 1

PAGE 28 OF 30

TOPIC : ADVANCED LINUX SHELL SCRIPTING BASICS

Small steps daily.
Big skills for life!

Keep Practicing !



1. Functions with Return Values

```
function add() {  
    local a=$1  
    local b=$2  
    local sum=$((a+b))  
    echo $sum  
}  
result=$(add 5 7)  
echo "Sum is: $result"
```

Output
Sum is: 12

💡 Tip: Functions can return values using echo or exit.

2. Here Strings (Advanced)

```
str="DevOps is awesome"  
echo ${str:4}      # Ops is awesome (from index 4)  
echo ${str:0:7}    # DevOps is  
echo ${str:-7}     # awesome (last 7 chars)  
echo ${str// /_}   # DevOps_is_awesome (replace space)  
echo ${#str}       # 19 (length of string)
```

💡 Tip: Substring, length, replace are powerful string tools.

3. Arrays - Advanced Operations

```
arr=(10 20 30 40 50)  
echo "All: ${arr[@]}"  
echo "First: ${arr[0]}"  
echo "Last: ${arr[-1]}"  
echo "Slice (1-3): ${arr[@]:1:3}"  
arr[2]=99 # Update index 2  
echo "Updated: ${arr[@]}"
```

Output
All: 10 20 30 40 50
First: 10
Last: 50
Slice (1-3): 20 30 40
Updated: 10 20 99 40 50

💡 Tip: Arrays are zero-indexed. -1 refers to last element.

4. Associative Arrays (Key-Value)

```
declare -A user  
user["name"]="Shalom"  
user["role"]="DevOps Engineer"  
user["id"]="101"  
echo "Name: ${user[name]}"  
echo "Role: ${user[role]}"  
echo "ID: ${user[id]}"
```

Output
Name: Shalom
Role: DevOps Engineer
ID: 101

💡 Tip: Associative arrays use keys instead of indexes.

5. String Manipulation - Advanced

```
str=" Learn Shell Scripting "  
echo "${str##* }" # Remove leading spaces -> "Learn Shell Scripting "  
echo "${str%%* }" # Remove all leading spaces -> "Scripting "  
echo "${str%*}"   # Remove last word -> "Learn Shell"  
echo "${str,,}"   # Convert to lowercase -> " learn shell scripting "  
echo "${str^^}"   # Convert to uppercase -> " LEARN SHELL SCRIPTING "
```

💡 Tip: # removes shortest match, ## removes longest match.

6. Case Statement - Advanced (Pattern Matching)

```
read -p "Enter OS (linux/windows/mac): " os  
case "$os" in  
    linux|Linux) echo "Linux selected" ;;  
    windows|Windows) echo "Windows selected" ;;  
    mac|Mac) echo "Mac selected" ;;  
    *) echo "Unknown OS" ;;  
esac
```

Output (Example)
Enter OS (linux/
windows/mac): linux
Linux selected

💡 Tip: Patterns like linux|Linux match multiple values.

7. Arithmetic & Comparison in Shell

```
a=10; b=3  
echo "Sum: $((a+b))" # 13  
echo "Diff: $((a-b))" # 7  
echo "Mul: $((a*b))" # 30  
echo "Div: $((a/b))" # 3  
echo "Mod: $((a%b))" # 1
```

```
if [ $a -gt $b ]; then  
    echo "a is greater"  
elif [ $a -lt $b ]; then  
    echo "b is greater"  
else  
    echo "Both are equal"  
fi
```

Output
Sum: 13
Diff: 7
Mul: 30
Div: 3
Mod: 1
a is greater

💡 Tip: Use (()) for arithmetic and [] for comparisons.

8. Read with Timeout

```
read -t 5 -p "Enter your name: " name  
if [ $? -eq 0 ]; then  
    echo "Hello, $name"  
else  
    echo "Timeout! No input received"  
fi
```

Output
If input in 5 sec
→ Hello, Alice
Else
→ Timeout! No
input received

💡 Tip: -t sets timeout in seconds. \$? is exit status.

9. File Handling - Read Line by Line

```
while IFS= read -r line; do  
    echo "Line: $line"  
done < file.txt
```

Output (file.txt)
Line: DevOps
Line: is
Line: Powerful

💡 Tip: Use < to read from files line by line safely.

★ Quick Tips

- ✓ Quotes preserve spaces.
- ✓ Use variables to avoid hardcoding.
- ✓ Test scripts in small steps.
- ✓ Use comments and echo for debugging.



DEVOPS 60 DAYS

ZERO TO HERO SERIES

DAY 1

PAGE 29 OF 30

TOPIC : ADVANCED LINUX SHELL SCRIPTING BASICS

Small steps daily.
Big skills for life!

Keep Practicing !



1. Command Line Arguments (Advanced)

Parameter	Meaning	Example
\$0	Script name	\$0 → script.sh
\$1 .. \$9	First 9 arguments	\$1 → apple
\$#	Total number of args	\$# → 3
\$@	All arguments as list	\$@ → a b c
\$*	All arguments as string	\$* → "a b c"
\$\$	Current PID	\$\$ → 12345
\$?	Last exit code	\$? → 0

💡 Tip: Use "\$@" to handle arguments with spaces.

5. Find, Grep and AWK in Scripts

```
# Find all .log files in /var/log
find /var/log -name "*.log"
```

```
# Search keyword in a file
grep "ERROR" app.log
```

```
# Extract 2nd column using awk
awk '{print $2}' data.txt
```

data.txt Content:

```
ID Name Role
101 Shalom DevOps
102 Alice Developer
103 Bob Tester
```

Output (awk):

```
Name
Shalom
Alice
Bob
```

💡 Tip: Combine tools for powerful automation.

2. Reading from User with Prompt

```
read -p "Enter your name: " NAME
echo "Hello, $NAME!"

read -s -p "Enter password: " PASS
echo "Password length: ${#PASS}"
```

Sample Run:

```
Enter your name: Shalom
Hello, Shalom!
Enter password:
Password length: 8
```

💡 Tip: -s hides input (silent mode).

6. Backup Script Example

```
#!/bin/bash
SRC=/etc
DEST=/backup/etc_$(date +%F).tar.gz
tar -czf $DEST $SRC
echo "Backup done: $DEST"
```

Sample Run:

Backup done:
/backup/etc_2025-05-24.tar.gz

💡 Tip: Always take backups before major changes.

3. Using Select Loop

```
select opt in Start Stop Restart Exit
do
  case $opt in
    Start) echo "Starting...";;
    Stop) echo "Stopping...";;
    Restart) echo "Restarting...";;
    Exit) break;;
    *) echo "Invalid option";;
  esac
done
```

Sample Run:

```
1) Start
2) Stop
3) Restart
4) Exit
#? 2
Stopping...
```

💡 Tip: select is menu driven and user friendly.

7. Log File Rotation (Simple)

```
LOGFILE="app.log"
MAXSIZE=1000
if [ -f $LOGFILE ]; then
  size=$(stat -c%s $LOGFILE)
  if [ $size -gt $MAXSIZE ]; then
    mv $LOGFILE ${LOGFILE}.old
    touch $LOGFILE
  fi
fi
```

How it works:

- Check log size
- If > MAXSIZE
 - Rename old log
 - Create new log

💡 Tip: Helps to manage log file size automatically.

4. Trap Command for Signal Handling

```
cleanup() {
  echo "Cleaning up..."
}
trap cleanup SIGINT SIGTERM
echo "Script running. Press Ctrl+C to exit."
while true; do sleep 5; done
```

Signals:

SIGINT → Ctrl+C

SIGTERM → Terminate

SIGKILL → Cannot trap

8. Real Time Script Execution with Watch

```
# Install: sudo apt install watch
watch -n 5 'df -h'
```

What it does?

Runs command every
5 seconds and
shows output live.

💡 Tip: Great for monitoring disk, memory, processes etc.

DEVOPS 60 DAYS

ZERO TO HERO SERIES

DAY 1

PAGE 30 OF 30

TOPIC : ADVANCED LINUX SHELL SCRIPTING BASICS

Small steps daily.
Big skills for life!

Keep Practicing !



1. Schedule Script with Cron

```
# Edit crontab
crontab -e

# Run script every day at 2 AM
0 2 * * * /home/user/backup.sh

# Run script every Sunday at 6 PM
0 18 * * 0 /home/user/report.sh
```

Cron Fields:

```
* * * * *
| | | | |
| | | +-- Day of week
| | +--- Month
| +----- Day of month
+----- Hour
+----- Minute
```

Tip: Cron is great for automation and scheduling.

5. Redirecting Output

```
echo "Hello" > output.txt # Overwrite
echo "World" >> output.txt # Append
ls -l > files.txt # Save output
ls -l 2> error.log # Save errors
command > out.txt 2>&1 # Save both
```

Sample Run:

out.txt → Normal output
error.log → Errors
out.txt → All output

Tip: 2>&1 redirects error to the same destination as output.

2. Functions with Parameters

```
greet_user() {
    local name=$1
    local role=$2
    echo "Hello $name, You are a $role!"
}

greet_user "Shalom" "DevOps Engineer"
greet_user "Alice" "Developer"
```

Sample Run:

```
Hello Shalom,
You are a DevOps
Engineer!

Hello Alice,
You are a Developer!
```

Tip: Functions become more powerful with parameters.

6. Trap for Error Handling

```
trap 'echo "Something went wrong!"' ERR
trap 'echo "Script interrupted!"' INT

echo "Running script..."
false # This will trigger ERR
echo "This will not run"
```

Sample Run:

Running script...
Something went
wrong!

Tip: Traps help handle errors and interruptions gracefully.

3. Working with Files and Directories

```
DIR="/home/user/test"
mkdir -p $DIR
echo "Data" > $DIR/data.txt
if [ -f $DIR/data.txt ]; then
    echo "File exists"
fi

rm -rf $DIR # Remove directory
```

Sample Run:

```
File exists
(Directory removed)
```

Tip: Always use quotes with variables.

7. While Read Loop (Process File Line by Line)

```
while IFS= read -r line; do
    echo "Line: $line"
done < data.txt
```

data.txt:

```
Apple
Banana
Cherry
```

Sample Run:

```
Line: Apple      Line: Banana      Line: Cherry
```

Tip: Great for processing large files line by line.

4. Case Statement with Wildcards

```
read -p "Enter filename: " file
case "$file" in
    *.sh) echo "Shell Script" ;;
    *.txt) echo "Text File" ;;
    *.log) echo "Log File" ;;
    *)    echo "Unknown File Type" ;;
esac
```

Sample Run:

```
Enter filename: script.sh
Shell Script
```

8. Script Template (Best Practice)

```
#!/bin/bash
# Script Name: backup.sh
# Description: Backup important data
# Author: ShalomInfoSoft
# Date: 2025-05-24

set -euo pipefail
echo "Backup started..."
# Your logic here
echo "Backup completed successfully."
```

Tip:

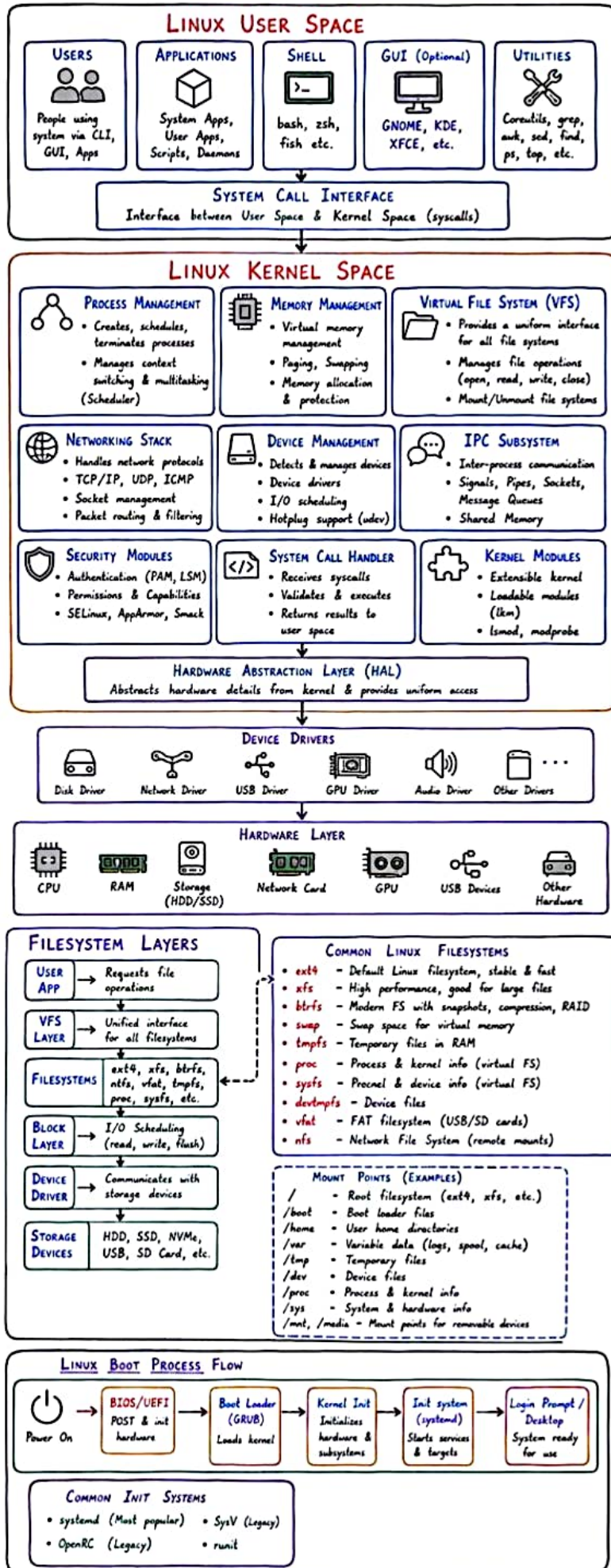
- ✓ Use comments
- ✓ Use set -euo
- ✓ Use meaningful variable names
- ✓ Handle errors
- ✓ Give proper output



Congratulations on completing
Day 1 - Advanced Linux Shell Scripting Basics!
Keep practicing, keep learning, and keep growing.
You are one step closer to becoming a **DevOps Expert!**



High level view of Linux Architecture and how all components work together.

**EXPLANATION (COMPONENT BY COMPONENT)**

- USERS / CLIENTS**
 - Humans interact with Linux using CLI, GUI or applications.
 - Everything starts from user requests.
- APPLICATIONS**
 - System programs, services, daemons, and user applications.
 - Examples: Firefox, Nginx, Docker, VS Code, etc.
- SHELL**
 - Command line interface to execute commands & scripts.
 - Acts as a bridge between user & kernel via syscalls.
- SYSTEM CALL INTERFACE**
 - User space programs request services from the kernel.
 - Syscalls provide a secure & controlled interface.
- PROCESS MANAGEMENT**
 - Kernel schedules processes using algorithms (CFS, etc.).
 - Manages lifecycle, priorities, context switching.
- MEMORY MANAGEMENT**
 - Manages RAM using virtual memory, paging & swapping.
 - Ensures isolation & protection between processes.
- VIRTUAL FILE SYSTEM (VFS)**
 - Provides a unified way to access different filesystems.
 - Supports various FS types with same system calls.
- NETWORKING STACK**
 - Implements network protocols (TCP/IP, UDP, ICMP).
 - Handles connections, routing, firewall (netfilter/iptables).
- DEVICE MANAGEMENT**
 - Detects hardware and loads appropriate drivers.
 - Manages devices as files in /dev using udev.
- IPC SUBSYSTEM**
 - Allows processes to communicate with each other.
 - Mechanisms: Signals, Pipes, Sockets, Shared Memory.
- SECURITY MODULES**
 - Enforces security policies and access control.
 - Includes PAM, SELinux, AppArmor, Capabilities.
- SYSTEM CALL HANDLER**
 - Validates, executes syscalls & returns results.
 - Core of the kernel's interface with user space.
- KERNEL MODULES**
 - Extend kernel functionality without reboot.
 - Examples: virtualbox, nvidia, ebttables, etc.
- HARDWARE ABSTRACTION LAYER (HAL)**
 - Provides standard interface for kernel to interact with hardware.
 - Hides hardware complexity.
- DEVICE DRIVERS**
 - Kernel modules that operate specific hardware.
 - Communicates with hardware via HAL.
- HARDWARE LAYER**
 - Physical components of the system.
 - CPU executes instructions, RAM stores data, etc.
- FILESYSTEM LAYERS**
 - VFS abstracts filesystem operations.
 - Block layer handles I/O requests efficiently.
 - Data is read/written to storage devices.
- BOOT PROCESS**
 - System boots from firmware → bootloader → kernel.
 - init system starts services and brings system up.
- INIT SYSTEMS**
 - Manages system services & daemons.
 - Systemd is most widely used in modern distros.

KEY COMPONENTS SUMMARY

- **Red** - Core kernel components (critical for system operation)
- **Blue** - User space & system interaction components
- **Green** - Filesystem & storage related components
- **Purple** - Hardware & low-level components
- **Orange** - Boot process & system initialization